

Class 6: R Functions

Melissa (PID: A19149673)

Table of contents

Background	1
Q1. Write your first R function: <code>add()</code>	1
Q2. Write a <code>generate_dna()</code> function	3
Q3. Write a <code>generate_protein()</code> function	6
Q4. Generate random protein sequences of length 6 to 13	7
Q5. BLASTp search against nr — are your peptides “unique in nature”?	8
Then answer the following in 1–3 sentences each:	8
Q6. Connecting your findings to immunology (MHC class II and T-cell activation)	9

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call a function)
- input *arguments* (one or more comma separated inputs that go inside the brackets where when we call the function),
- the *body* (the lines of R code that do the work of the function).

Q1. Write your first R function: `add()`

Here we will create a function to add some numbers. Let’s call it `add()`.

Input arguments can be either **required** or **optional**. The later have fall-back default values that will be used if the user does not specify them.

Q1a: Your first version of the function should add two input numbers together. For example, `add(4,7)` should return 11. [1 pt]

```
# This defines a function called add() that adds two numbers together
# x is the first number and required and y is optional
# setting the y to 0 allows the function to still run if just one input is given
# Return the sum of x and y after
add <- function(x,y=0) {
  x + y
}
```

```
add(4, 7)
```

```
[1] 11
```

The function `add()` is used to add two numbers. The `x` is also a required input being our first number and `y` is optional with a default value of 0. This essentially allows the function to still run even if only one input is provided and the function returns the sum of `x` and `y`.

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [1 pt]

```
# The updated add() function allows to work with either,
# two separate inputs or one input vector
# the sum() is used since it can add multiple values together as a result
# the sum() adds all values from x and y
add <- function(x,y=0) {
  sum(x,y)
}
```

```
add( c(4,7,10) )
```

```
[1] 21
```

The second version uses `sum()` instead of `x + y` as this makes the function more flexible and to add two separate numbers such as `add(c(4,7,10))` as this improves the function since it is able to handle more than one type of input format as a result.

Q1c: c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)`. (3 dots)

```
# The third version of add() is using the 3 dots, ...
# as the 3 dots argument allows the user to pass any number of inputs
# The sum() will then add all values given
# Then return the sum of all arguments are passed into the function
add <- function(...) {
  sum(...)
}
```

The third version is using the ..., which is called the dots argument, allows any number of inputs to be entered into the function. This makes the function even more flexible as it can accept individual numbers, vectors as well as separate arguments. The sum() function is then used to add everything together and returns the total.

```
add(1, 2, 3, -4)
```

```
[1] 2
```

Q2. Write a generate_dna() function

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call

A useful function here is the “base R” sample() function:

```
sample(1:5, size=3)
```

```
[1] 1 5 4
```

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 5 2 2 3 4 5 5 1 1 1 5 2 2 5 3 3 1 4 1 1 2 4 4 1 4 3 2 1 2 4 2 5 1 2 1 3 5 2
[39] 1 5 2 4 4 1 5 1 1 3 2 5 3 1 2 1 5 1 3 5 3 2
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A","C","G","T"), size=10, replace = TRUE)
```

```
[1] "T" "G" "C" "G" "T" "C" "C" "G" "G" "A"
```

Q2a: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
# The function is used to generate a random DNA sequence
# The len specifies the desired length of the DNA sequence
# the sample() randomly selects nucleotides from ACGT
# replace set equal to TRUE allows nucleotides to repeat
generate_dna <- function(len=10) {
  sample(x=c("A","C","G","T"), size=len, replace = TRUE)
}
```

```
generate_dna()
```

```
[1] "T" "C" "A" "G" "G" "G" "T" "T" "T" "T"
```

```
generate_dna(len=100)
```

```
[1] "T" "G" "C" "A" "C" "T" "T" "T" "C" "G" "T" "A" "T" "A" "T" "G" "T" "G"
[19] "A" "C" "G" "G" "A" "C" "G" "C" "T" "C" "G" "G" "C" "C" "A" "T" "T" "G"
[37] "T" "C" "A" "G" "T" "T" "A" "G" "G" "C" "A" "A" "C" "C" "G" "T" "G" "C"
[55] "T" "C" "C" "T" "C" "G" "T" "T" "A" "A" "C" "G" "A" "G" "G" "T" "A" "C"
[73] "A" "T" "C" "A" "G" "A" "C" "A" "A" "G" "T" "G" "G" "A" "A" "T" "G" "A"
[91] "C" "T" "G" "C" "C" "T" "C" "T" "C" "A"
```

The function generates a random DNA sequence of a specified length using the `sample()` function as this essentially randomly selects nucleotides from the set of ACGT. The argument of the `replace` equal to `TRUE` allows nucleotides to repeat and is vital since real DNA sequences have repeated bases. Additionally, the output is a vector of individual nucleotide characters.

Q2b: Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

```
# The updated function is to optionally return a single DNA string
# the single.element is set to FALSE as this means return a vector and if it was set to TRUE
# it would collapse the vector into one string instead
generate_dna <- function(len = 10, single.element = FALSE) {
  # random nucleotides generated below
```

```

ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)

#cat("Hello...")

if(single.element) {
  #cat("Is it me you are looking for...")
  ans <- paste(ans, collapse = "")
}

return(ans)
}

```

```
generate_dna(6)
```

```
[1] "A" "C" "G" "C" "C" "T"
```

```
generate_dna(7, single.element=TRUE)
```

```
[1] "CCGAGAA"
```

This version allows two types of output and by default it would return a vector of nucleotides, but if the `single.element` was `TRUE`, the function can use `paste(..., collapse="")` to combine all elements into a single DNA string.

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

Q2c: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. Be sure to include clear comments that explain each step of your function. [2 pts]

```

# The final version that prints DNA in FASTA format includes a header line
# which starts with ">" and includes sequence info
# the FASTA format also includes the DNA sequence on the next line
generate_dna <- function(len = 10, single.element = FALSE) {
  # once again random nucleotides generated
  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)

  # this collapses into one string if needed
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }
}

```

```

### Format as FASTA with an ID line
# prints FASTA header that has sequence length
cat( paste(">len", len, "\n", sep=""))
# prints DNA sequence
cat(ans)
cat("\n")
##

return(ans)
}

```

```
x <- generate_dna(11)
```

```

>len11
A C T G C A G C T T A

```

The final version formats the DNA sequence in FASTA format which begins with > and includes the sequence length and that is followed by the DNA sequence on the next line. The cat() function is utilized instead of the print() to be able to control formatting as well as ensure appropriate break lines.

Q3. Write a generate_protein() function

Q3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG". [2 pts]

```

# the function is used to generate a random protein sequence
# and len specifies the length of the protein sequence
# the aa contains the 20 common amino acids
# the sample() function randomly selects amino acids
# the replace once again set to TRUE allows proteins to repeat amino acids
generate_protein <- function(len=9) {
  aa <- c("A", "R", "N", "D", "C", "E", "Q",
          "G", "H", "I", "L", "K", "M", "F",
          "P", "S", "T", "W", "Y", "V")
  # amino acids randomly sample
  ans <- (sample(x=aa, size=len, replace = TRUE))
  # shown to collapse into a single protein string
  paste (ans, collapse = "")
}

```

```
}
```

```
generate_protein(6)
```

```
[1] "WLSIQE"
```

Q4. Generate random protein sequences of length 6 to 13

Q4: Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for (l in 6:13) {  
  cat(">id.", l, "\n", sep="")  
  cat(generate_protein(l), "\n", sep="")  
}
```

```
>id.6  
KCLEVI  
>id.7  
YTLWIYI  
>id.8  
TYCDHKHD  
>id.9  
SHVPLVDAV  
>id.10  
CKDQEAGWTL  
>id.11  
CWYGSCLNGHD  
>id.12  
EYNYQCDAQQIT  
>id.13  
TQKIWTYSSWFMN
```

Q5. BLASTp search against nr — are your peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best Hit %Identity	%Coverage	Unique? (N/Y)
6	100	100	N
7	100	100	N
8	100	88	Y
9	88	100	Y
10	83	100	Y
11	100	82	Y
12	100	67	Y
13	83	92	Y

The table results above display a clear transition of very short peptides being in between 6 to 7 amino acids, as they are easily found in known proteins in comparison to peptides of length 8 and above, which appear increasingly unique.

Then answer the following in 1–3 sentences each:

- a. At which sequence length do your randomly generated peptides start to look “unique in nature”(i.e. no 100% coverage + 100% identity hit)? [1 pt]

My randomly generated peptides start to look “unique in nature” at length 8 since this is the first length where there is no hit with both 100% identity and 100% query coverage. According to my table above, lengths 6 and 7 had perfect matches of 100% coverage and 100% identity hit, but length 8 no longer did as it obtained 100% identity and 88% coverage.

- b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe. [1 pt]

Very short random peptides are almost always found in the nr database since the number of possible sequences is still relatively small at short lengths in comparison to the known protein universe which is large as the number of possible peptides grows as 20^L to the power of L . Hence, as the peptide length increases, the sequence space expands pretty quickly and random sequences become much less likely to have already been observed. Moreover, this is why short peptides match existing proteins while the longer peptides are more prone to appear unique.

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

According to my Q5 results, the minimum length is most likely around 8 amino acids or longer since very short peptides are a bad design choice for the immune system since they are so common in already existing proteins that would frequently match “self” sequences and would not provide enough specificity. Additionally, as peptide length increases, the number of possible sequences continuously grows exponentially which makes longer peptides to be more informative which not only helps the immune system avoid wasting responses on short, non specific peptide fragments but can help improve detection of pathogens overall.